

In SQL ist es wichtig, zwischen der **Syntax-Reihenfolge** (wie du den Code schreibst) und der **logischen Ausführungsreihenfolge** (wie die Datenbank die Abfrage intern abarbeitet) zu unterscheiden.

Hier ist die korrekte syntaktische Reihenfolge für SQLite und die meisten anderen SQL-Dialekte:

1. Die syntaktische Reihenfolge (Schreibweise)

Du musst die Keywords in dieser festen Reihenfolge notieren, sonst wirft der Parser einen Fehler:

1. **SELECT** (und optional **DISTINCT**)
2. **FROM**
3. **JOIN** (mit **ON**)
4. **WHERE**
5. **GROUP BY**
6. **HAVING**
7. **ORDER BY**
8. **LIMIT / OFFSET**

Merksatz (Eselsbrücke): “*Sweety, Find Joey With Great Help Or Luck*” (Select, From, Join, Where, Group, Having, Order, Limit)

2. Die logische Ausführungsreihenfolge (Reihenfolge der Verarbeitung)

Interessanterweise arbeitet die Datenbank das Statement nicht von oben nach unten ab. Das zu verstehen, hilft enorm beim Debugging (z. B. warum man Alias-Namen aus dem SELECT oft nicht im WHERE nutzen kann).

1. **FROM / JOIN:** Zuerst wird die Datenquelle bestimmt (und Tabellen verknüpft).
2. **WHERE:** Zeilen werden gefiltert, bevor gruppiert wird.
3. **GROUP BY:** Die verbleibenden Zeilen werden zusammengefasst.
4. **HAVING:** Die Gruppen werden gefiltert.
5. **SELECT:** Die Spalten werden ausgewählt (und Aggregate berechnet).
6. **DISTINCT:** Duplikate werden entfernt.
7. **ORDER BY:** Das Ergebnis wird sortiert.
8. **LIMIT:** Die Anzahl der Zeilen wird beschnitten.

Ein typischer Stolperstein für Schüler:

Im WHERE kann man keine Spalten-Aliase verwenden, die man erst im SELECT definiert hat, weil das WHERE ausgeführt wird, **bevor** das SELECT die Spalten benennt.

Beispiel:

```
-- Das funktioniert in vielen SQL-Dialekten NICHT:
SELECT salary * 1.1 AS new_salary
FROM employees
WHERE new_salary > 5000; -- Fehler: new_salary unbekannt
```

```
-- In SQLite funktioniert es oft aus Kulanz, aber rein logisch
-- müsste man die Berechnung im WHERE wiederholen oder eine Subquery/CTE nutzen.
```

In deinen bevorzugten ORMs wie **Drizzle** oder **Prisma** wird diese Reihenfolge durch die Methoden-Verkettung (Fluent API) meistens intuitiv korrekt abgebildet, da die Library den SQL-String am Ende für dich baut.